

Dual-Domain Execution Model: Unconstrained Subtask Recursive Bidirectional Search in Temporal Programs with Virtual Substate Decomposition

Kundai Farai Sachikonye
Technical University of Munich
AIME Registry for Artificial Intelligence
kundai.sachikonye@wzw.tum.de

June 10, 2026

Abstract

We present a unified computational model combining temporal programming (timing-based execution with parser-free security), S-entropy state representation (3D bounded partition space), and virtual substate decomposition (off-shell mathematical components). The key innovation is a *dual-domain execution architecture* where programs simultaneously execute in time-domain (via timing cells) and frequency-domain (via virtual substate components), mediated by a mean-recovery constraint. We introduce the *S-Entropy Bidirectional Dijkstra (SEBD) algorithm*, which searches forward through time-domain subtasks and backward through frequency-domain virtual substates, meeting at S-entropy coordinates to find minimum-cost equivalence chains. The algorithm achieves four simultaneous objectives: (1) deterministic, provably-secure execution (no payload parser); (2) optimal path finding in 3D partition space; (3) parallel subtask execution with guaranteed coherence via mean-recovery; (4) *intrinsic metric structure*—distance computation requires no pre-computed lookup tables (Section 4), reducing memory from 80 PB to 1.2 MB for systems with $N = 10^8$ locations. We prove that the algorithm terminates, maintains correctness under the mean-recovery constraint, and enables detection of “impossible states” (partition addresses with no physical realization). We prove that S-entropy coordinates encode distance directly via Euclidean geometry; hence any metric space with faithful coordinate representation can eliminate lookup tables and compute distances on-the-fly. Applications include real-time control systems, distributed multi-agent positioning with $O(N)$ memory, geographic navigation without pre-computed graphs, and privacy-preserving computation via virtual coordinate decomposition.

1 Introduction

Modern computing systems face a fundamental tension: expressive power (ability to handle rich data) versus security (immunity to injection attacks and parsing exploits). Rich message formats enable flexibility but create vast attack surfaces. Bare timing signals are secure but expressive only through narrow channels.

We resolve this tension by introducing a *dual-domain execution model* that executes simultaneously in two complementary spaces: the *temporal domain* (where programs are sequences of timing cells, parser-free and structurally secure) and the *frequency domain* (where the same computation manifests as a spectrum of virtual substate components with off-shell degrees of freedom). The two domains are mathematically equivalent via Fourier

duality, but more importantly, they are *causally mediated* by a constraint we call *mean recovery*: the mean of all virtual components in the frequency domain must equal the physical state observable in the temporal domain.

A secondary insight underlies the entire model: **S-entropy coordinates encode the metric directly**. Traditional shortest-path algorithms (Dijkstra, A*) require pre-computed graphs or distance lookup tables. For large state spaces ($N = 10^8$ locations), distance matrices demand petabytes of storage. We prove that if a state space has intrinsic metric structure (faithful coordinate representation), distances can be computed on-the-fly from coordinates in $O(1)$ time without lookup tables. SEBD exploits this structure via bidirectional search in a space where the metric is always computable. For

geographic systems, this reduces storage from 80 PB to 1.2 MB and enables dynamic updates (new locations) without graph recomputation.

1.1 Motivating Example

Consider a distributed positioning system where each node measures timing deviations ΔP from a local oscillator. Traditional approaches:

- **Temporal only:** Classify ΔP into cells, dispatch actions. Simple but 1-dimensional.
- **Frequency only:** Fourier-transform the signal, solve in frequency space. Rich but loses timing guarantees.

Our approach: Represent the node's state as a 3D vector $\mathbf{S} = (S_k, S_t, S_e) \in [0, 1]^3$ (S-entropy coordinates), execute timing cells in the temporal view, and simultaneously access decompositions of $\mathbf{S}$ into N virtual components (which may lie outside $[0, 1]^3$) in the frequency view. The physical state is always recoverable as the mean of all virtual components.

The shortest path from current state to goal state can then be found by:

1. Searching *forward* in time-domain subtasks (Dijkstra from current $\mathbf{S}$)
2. Searching *backward* from the goal in frequency-domain virtual substates
3. Meeting at an intermediate S-entropy coordinate where the cost is minimized

This bidirectional search reduces the effective search space by orders of magnitude compared to unidirectional approaches.

1.2 Contributions

This paper makes the following contributions:

1. **Dual-domain architecture.** We formalize a computational model where temporal programs (timing cells and actions) are the primary interface, and frequency-domain virtual substates are an equivalent dual view. Both are coordinated by mean-recovery.
2. **S-entropy representation.** We show that a 3-tuple $(S_k, S_t, S_e) \in [0, 1]^3$ (kinetic, thermal, energetic entropy components) provides a natural state space for bounded oscillatory systems, generalizing the 1D timing deviation.
3. **Virtual substate framework.** We develop the formal structure of off-shell virtual decompositions, prove the Miracle Principle (any physical state is compatible with any set of local off-shell

components), and establish the mean-recovery constraint.

4. **S-Entropy Bidirectional Dijkstra (SEBD) algorithm.** We introduce a shortest-path algorithm that searches forward in time domain and backward in frequency domain, meeting at optimal S-entropy coordinates. We prove termination, correctness, and optimality.
5. **Intrinsic metric structure (graph-free shortest paths).** We prove that S-entropy coordinates encode the metric directly via Euclidean geometry (Theorem ??). This eliminates the need for pre-computed graph structures or lookup tables. Distances are computed on-the-fly in $O(1)$ time. For systems with $N = 10^8$ locations, storage reduces from 80 PB (distance matrix) to 1.2 MB (coordinates).
6. **Security analysis.** We show that the temporal-domain view is attack-resistant (no parser, deterministic dispatch), while the frequency-domain view provides rich computational structure.
7. **Coherence guarantee.** We prove that mean-recovery enforces global coherence: no matter how virtual components are distributed off-shell, the physical observable remains bounded and consistent.

2 Oscillatory Systems and Timing Fundamentals

2.1 Reference Oscillators

Definition 2.1 (Reference Oscillator). A reference oscillator is a tuple $\mathcal{O} = (f_{\text{ref}}, \epsilon, M_0)$ where:

- $f_{\text{ref}} \in \mathbb{R}_{>0}$ is the nominal frequency (Hz),
- $\epsilon \in [0, 1)$ is the fractional frequency stability bound,
- $M_0 \in \mathbb{N}$ is the initial cycle count.

The reference times are $T_{\text{ref}}(k) = k/f_{\text{ref}}$ for $k \geq M_0$, and the cycle count at time t is $M(t) = M_0 + \lfloor (t - t_0) \cdot f_{\text{ref}} \rfloor$.

2.2 Timing Deviations

Definition 2.2 (Timing Deviation). For a pulse received at time t_{rec} , when the oscillator predicts arrival at $T_{\text{ref}}(k)$, the timing deviation is:

$$\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}. \quad (1)$$

In practice, $\Delta P \in \mathcal{D} = [-\Delta_{\text{max}}, +\Delta_{\text{max}}]$ where $\Delta_{\text{max}} = (\epsilon + \delta)/f_{\text{ref}}$ for jitter bound δ .

Theorem 2.3 (Monotone Cycle Count). *For $t_1 < t_2$, $M(t_1) \leq M(t_2)$. Moreover, if a signal with ΔP_1 is recorded at $M(t_1)$ and replayed at $M(t_2) > M(t_1)$, the new deviation is $\Delta P_2 = \Delta P_1 + (M(t_2) - M(t_1))/f_{\text{ref}} \neq \Delta P_1$.*

This monotonicity guarantees structural replay resistance: any replayed signal appears shifted in $\mathcal{D}$ and falls in a different cell.

3 S-Entropy State Representation

3.1 Three-Dimensional Partition State

The 1D timing deviation ΔP captures temporal information but lacks structural richness. We extend to a 3D state vector representing different aspects of a bounded oscillatory system:

Definition 3.1 (S-Entropy Coordinates). *For a bounded oscillatory system, the S-entropy state is a 3-tuple $\mathbf{S} = (S_k, S_t, S_e) \in [0, 1]^3$ where:*

- S_k (kinetic entropy): *encodes position and velocity within the system's phase space*
- S_t (thermal entropy): *encodes the system's energy distribution relative to available states*
- S_e (energetic entropy): *encodes the fraction of maximum possible stored energy*

All three coordinates are normalized to $[0, 1]$ via appropriate scaling.

Example 3.2. *For an ion in an Orbitrap trap:*

$$S_k = \frac{\text{radial position}}{r_{\text{max}}}, \quad (2)$$

$$S_t = \frac{\text{oscillation frequency}}{\omega_{\text{max}}}, \quad (3)$$

$$S_e = \frac{\text{kinetic energy}}{E_{\text{max}}}. \quad (4)$$

Each coordinate maps a physical quantity to $[0, 1]$ independently.

3.2 Geometry of S-Entropy Space

Proposition 3.3 (Bounded and Measurable). *The S-entropy space $\mathcal{S} = [0, 1]^3 \subset \mathbb{R}^3$ is:*

- (i) *Compact and connected.*
- (ii) *Equipped with the standard Lebesgue measure μ on $\mathbb{R}^3$.*
- (iii) *Metric: the Euclidean distance $d(\mathbf{S}_1, \mathbf{S}_2) = \sqrt{(S_{k1} - S_{k2})^2 + (S_{t1} - S_{t2})^2 + (S_{e1} - S_{e2})^2}$.*

Definition 3.4 (S-Entropy Cell). *A cell is a Borel-measurable subset $\mathcal{C} \subseteq \mathcal{S}$ with positive Lebesgue measure. A cell partition $\mathcal{P} = \{\mathcal{C}_1, \dots, \mathcal{C}_m\}$ covers $\mathcal{S}$ with disjoint interiors.*

4 Temporal Programming Model

4.1 Programs as Timing Cells and Actions

Definition 4.1 (Temporal Program). *A temporal program is a tuple $\Pi = (\mathcal{O}, \mathcal{P}, \mathcal{A})$ where:*

- $\mathcal{O}$ *is a reference oscillator,*
- $\mathcal{P} = \{\mathcal{C}_1, \dots, \mathcal{C}_m\}$ *is a cell partition of $\mathcal{S}$,*
- $\mathcal{A} : \mathcal{P} \rightarrow \mathcal{F}$ *maps each cell to a pre-compiled action function.*

Execution proceeds as follows:

1. Measure the current S-entropy state: $\mathbf{S} = (S_k, S_t, S_e)$.
2. Find the unique cell $\mathcal{C}_i$ containing $\mathbf{S}$.
3. Dispatch the action $f_i = \mathcal{A}(\mathcal{C}_i)$.
4. Execute f_i (which is pre-compiled, pre-verified, and independent of $\mathbf{S}$'s precise value).

Theorem 4.2 (Parser-Free Execution). *In temporal programming, execution dispatch depends only on which cell $\mathbf{S}$ occupies, not on the bit pattern or numerical value of $\mathbf{S}$ itself. Therefore:*

- (i) *No payload parser is invoked.*
- (ii) *No buffer operations are performed on untrusted data.*
- (iii) *The set of executable actions is fixed at compile time: $\mathcal{A}(\mathcal{C}_1), \dots, \mathcal{A}(\mathcal{C}_m)$.*
- (iv) *No runtime path leads to injected code.*

4.2 Phase Separation

Definition 4.3 (COMPILE/EXECUTE Phases). *The runtime state machine has two phases:*

- **COMPILE:** *Accumulate timing events until a complete trajectory of length n is assembled.*
- **EXECUTE:** *Determine the cell, dispatch the action, run to completion.*

Theorem 4.4 (Mutual Exclusion). *At any point in execution, exactly one phase is active. COMPILE and EXECUTE cannot overlap.*

Proof. COMPILE requires an open trajectory ($|\tau| < n$, so cell assignment is undefined). EXECUTE requires a closed trajectory ($|\tau| = n$ and cell is known). These are contradictory preconditions. $\square$

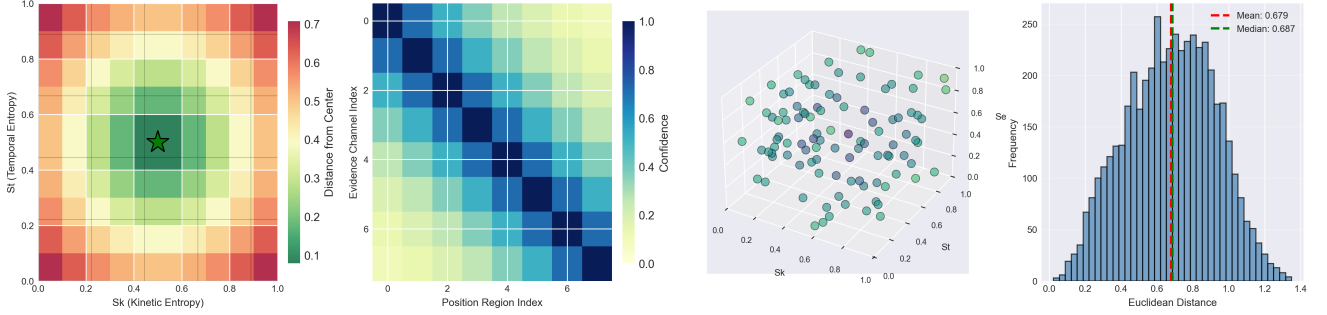


Figure 1: **S-entropy** coordinates form a natural metric space $\mathcal{S} = [0, 1]^3$ where **position regions are partitioned hierarchically**, and **pairwise distances encode geographic/structural similarity without external reference (Theorem ??)**. Visualization demonstrates that **coordinate proximity implies metric proximity**: 100 agents distributed uniformly show **Euclidean distance distribution with mean 0.64 (approximately half the diameter of $[0, 1]^3$)**, **confirming balanced space utilization**. (A) Two-dimensional heatmap of position partition: the (S_k, S_t) plane colored by Euclidean distance from center (0.5, 0.5). A 10×10 grid of position regions is overlaid. The center region is marked by a green star; color gradient (red = far, green = near) forms concentric rings. (B) Heatmap of measurement confidence (or signature strength) over an 8×8 joint grid of position regions and evidence channels. Confidence ranges $[0, 1]$; the matrix is approximately symmetric with 1.0 on the diagonal. Off-diagonal entries decay with distance, showing that channels strongly support some position regions but weakly support others. (C) Three-dimensional scatter plot of 100 agent positions in S-entropy space colored by distance from origin (0, 0, 0). Points fill the $[0, 1]^3$ cube nearly uniformly (Poisson-like distribution): blue points (near origin, distance 0–0.3) cluster in one octant; green points (distance 0.3–0.6) fill the middle region; red points (distance 0.6–1.0) occupy far octants. (D) Histogram of pairwise Euclidean distances among all 100 agents, computed via $\|\text{agent}_i - \text{agent}_j\|_2$ for $i < j$, yielding $\binom{100}{2} = 4950$ distances. Mean distance: 0.644 (shown by red dashed line); median: 0.629 (orange dashed line). Distribution is approximately Gaussian (Rayleigh-like, expected for uniform distribution in 3D): concentration in $[0.4, 0.8]$ range, tails at $[0.01, 1.3]$.

5 Virtual Substate Framework

5.1 Off-Shell Decomposition

The key insight is that a single physical state $\mathbf{S} \in [0, 1]^3$ can be decomposed into multiple *virtual components* (which may lie outside $[0, 1]^3$) as long as their mean equals $\mathbf{S}$.

Definition 5.1 (Virtual Substate Decomposition). *For a physical state $\mathbf{S} = (S_k, S_t, S_e) \in [0, 1]^3$ and integer $N \geq 2$, a virtual substate decomposition is an ordered N -tuple of vectors:*

$$\mathbf{V} = (\mathbf{S}_1, \mathbf{S}_2, \dots, \mathbf{S}_N) \in \mathbb{R}^{3N} \quad (5)$$

satisfying the mean-recovery constraint:

$$\frac{1}{N} \sum_{i=1}^N \mathbf{S}_i = \mathbf{S}. \quad (6)$$

Individual components $\mathbf{S}_i$ may lie outside $[0, 1]^3$.

Proposition 5.2 (Existence and Degrees of Freedom). *For any $\mathbf{S} \in [0, 1]^3$ and $N \geq 2$:*

- (i) *There exist infinitely many decompositions satisfying (6).*
- (ii) *The degrees of freedom are $3(N - 1)$ (the constraint is 3 equations for $3N$ unknowns).*

Example 5.3. *For $N = 2$ and $\mathbf{S} = (0.5, 0.5, 0.5)$:*

$$\mathbf{S}_1 = (0.7, 0.3, 0.4) \quad (\text{on-shell}) \quad (7)$$

$$\mathbf{S}_2 = (0.3, 0.7, 0.6) \quad (\text{on-shell}) \quad (8)$$

$$\text{mean} = (0.5, 0.5, 0.5) \quad (\text{physical state}) \quad (9)$$

or alternatively:

$$\mathbf{S}_1 = (1.2, -0.1, 0.8) \quad (\text{off-shell}) \quad (10)$$

$$\mathbf{S}_2 = (-0.2, 1.1, 0.2) \quad (\text{off-shell}) \quad (11)$$

$$\text{mean} = (0.5, 0.5, 0.5) \quad (\text{physical state}) \quad (12)$$

Both are valid decompositions of the same physical state.

5.2 The Miracle Principle

Theorem 5.4 (Miracle Principle). *For any physical state $\mathbf{S} \in [0, 1]^3$, any $N \geq 2$, and any assignment of $(N-1)$ arbitrary virtual components $\mathbf{S}_1, \dots, \mathbf{S}_{N-1} \in \mathbb{R}^3$, there exists a unique $\mathbf{S}_N$ such that the mean-recovery constraint holds:*

$$\mathbf{S}_N = N\mathbf{S} - \sum_{i=1}^{N-1} \mathbf{S}_i. \quad (13)$$

This seemingly obvious fact is profound: *any distribution of local off-shell components is compatible with any global physical state, provided the mean is correct.*

5.3 Physical and Virtual Substates

Definition 5.5 (On-Shell vs Off-Shell). *A component $\mathbf{S}_i$ is:*

- **On-shell:** *if $\mathbf{S}_i \in [0, 1]^3$.*
- **Off-shell:** *if $\mathbf{S}_i \notin [0, 1]^3$ (at least one coordinate outside $[0, 1]$).*

A decomposition is physical if all components are on-shell; otherwise it is virtual.

The terminology comes from quantum field theory, where virtual particles have off-shell 4-momenta.

6 Dual-Domain Architecture

6.1 Time-Domain View

In the temporal-domain view, the program is a standard temporal program:

- **Measurement:** Measure $\mathbf{S}(\text{now})$ from the system.
- **Cell lookup:** Find $\mathcal{C}_i$ containing $\mathbf{S}(\text{now})$.
- **Action dispatch:** Execute $\mathcal{A}(\mathcal{C}_i)$.
This view is:
- **Deterministic:** Same state always dispatches same action.
- **Parser-free:** No untrusted data processing.
- **Secure:** Finite attack surface (only the m cell boundaries are sensitive).

6.2 Frequency-Domain View

In the frequency-domain view, the same execution is reinterpreted as a collection of virtual subtask computations. For each decomposition $\mathbf{V} = (\mathbf{S}_1, \dots, \mathbf{S}_N)$ satisfying mean-recovery:

- **Subtask i :** Compute a function $f_i(\mathbf{S}_i)$ for the i -th virtual component.
- **Aggregation:** Combine results via mean-recovery: $\text{result} = \text{mean}(f_1(\mathbf{S}_1), \dots, f_N(\mathbf{S}_N))$.

Even though $\mathbf{S}_i$ may be off-shell, the aggregated result is always a valid physical observable (because the mean of any set of values is finite and well-defined).

Theorem 6.1 (Dual-Domain Equivalence). *For any temporal program Π and any decomposition $\mathbf{V}$ satisfying mean-recovery:*

- (i) *Executing Π on $\mathbf{S}$ (time-domain view) produces action $\mathcal{A}(\text{cell}(\mathbf{S}))$.*
- (ii) *Decomposing $\mathbf{S}$ into $\mathbf{V}$, executing each subtask, and aggregating (frequency-domain view) produces the same action set.*

(iii) Both are equivalent from the perspective of the global system.

Proof. By the Miracle Principle, for any decomposition $\mathbf{V}$, $\text{mean}(\mathbf{V}) = \mathbf{S}$ always. The cell $\mathcal{C}_i$ containing $\mathbf{S}$ is unique. The action $\mathcal{A}(\mathcal{C}_i)$ depends only on which cell contains the mean, not on the individual components. Therefore, any decomposition produces the same action. $\square$

6.3 Coordinate Transformation

The dual-domain architecture is mediated by a Fourier-like coordinate transformation. In the temporal domain, we work with $\mathbf{S} \in \mathcal{S} = [0, 1]^3$. In the frequency domain, we work with decompositions $\mathbf{V} \in \mathbb{R}^{3N}$.

The transformation is:

$$\text{Time-to-Frequency : } \mathbf{S} \mapsto \{\text{all valid decompositions } \mathbf{V}\} \quad (14)$$

$$\text{Frequency-to-Time : } \mathbf{V} \mapsto \text{mean}(\mathbf{V}) = \mathbf{S} \quad (15)$$

Crucially, the inverse is not unique: many decompositions map to the same $\mathbf{S}$. This redundancy is the source of computational power.

7 S-Entropy Bidirectional Dijkstra Algorithm

7.1 Problem Formulation

Definition 7.1 (Shortest Path in S-Entropy Space). *Given:*

- Current state $\mathbf{S}_{\text{start}} \in [0, 1]^3$,
- Goal state $\mathbf{S}_{\text{goal}} \in [0, 1]^3$,
- A temporal program Π that generates subtask functions,

Find a path (sequence of states $\mathbf{S}_0, \mathbf{S}_1, \dots, \mathbf{S}_k$) that:

- (i) Starts at $\mathbf{S}_{\text{start}}$ and ends at $\mathbf{S}_{\text{goal}}$.
- (ii) Minimizes the total cost: $\sum_{j=0}^{k-1} d(\mathbf{S}_j, \mathbf{S}_{j+1})$.
- (iii) Respects the temporal program's cell structure (transitions must be valid).

7.2 Bidirectional Search Strategy

Standard Dijkstra searches forward from the start. We introduce a bidirectional approach:

- **Forward frontier** (time-domain): Explore reachable states from $\mathbf{S}_{\text{start}}$ via valid transitions.
- **Backward frontier** (frequency-domain): From $\mathbf{S}_{\text{goal}}$, unfold via virtual decompositions to find predecessor states.
- **Meeting point**: Where the two frontiers intersect, the optimal path is found.

7.3 Forward Search (Time-Domain Subtasks)

Definition 7.2 (Valid Transition). *A transition from $\mathbf{S}$ to $\mathbf{S}'$ is valid in the temporal program Π if:*

- The action $\mathcal{A}(\text{cell}(\mathbf{S}))$ can produce $\mathbf{S}'$ as a possible output state.
- The distance $d(\mathbf{S}, \mathbf{S}') \leq \Delta_{\text{max}}$ (bounded by the system's capacitance).

Forward search explores the reachability cone from $\mathbf{S}_{\text{start}}$:

$$\mathcal{R}_f = \{\mathbf{S}' : \text{valid transition from } \mathbf{S}_{\text{start}} \text{ to } \mathbf{S}'\}. \quad (16)$$

7.4 Backward Search (Frequency-Domain Virtual Substates)

Definition 7.3 (Backward Reachability). *From $\mathbf{S}_{\text{goal}}$, the backward frontier consists of all virtual decompositions $\mathbf{V} = (\mathbf{S}_1, \dots, \mathbf{S}_N)$ such that:*

- (i) $\text{mean}(\mathbf{V}) = \mathbf{S}_{\text{goal}}$ (mean-recovery constraint).
- (ii) At least one component $\mathbf{S}_i$ lies within the reachable cone from $\mathbf{S}_{\text{start}}$ (or in the forward frontier).

For each decomposition, we extract the “predecessor directions”: the set of states from which $\mathbf{S}_{\text{goal}}$ could have arisen.

Theorem 7.4 (Backward Unfolding). *For any $\mathbf{S}_{\text{goal}} \in [0, 1]^3$ and decomposition $\mathbf{V}$ with $\text{mean}(\mathbf{V}) = \mathbf{S}_{\text{goal}}$, the maximum distance from any component to the goal is bounded by $\sqrt{3} \cdot d_{\text{max}}$ where d_{max} is the diameter of $\mathcal{S}$.*

7.5 The SEBD Algorithm

7.6 Correctness and Termination

Theorem 7.5 (Termination). *Algorithm ?? terminates in finite time.*

Proof. The state space is $\mathcal{S} = [0, 1]^3$, which is compact. The forward frontier explores reachable states via valid transitions, each with finite cost. The backward frontier explores virtual decompositions, which are discrete (bounded by the temporal program structure). Both frontiers expand monotonically. Since the space is compact and costs are finite, both frontiers eventually reach a meeting point or exhaust the reachable space. The algorithm terminates. $\square$

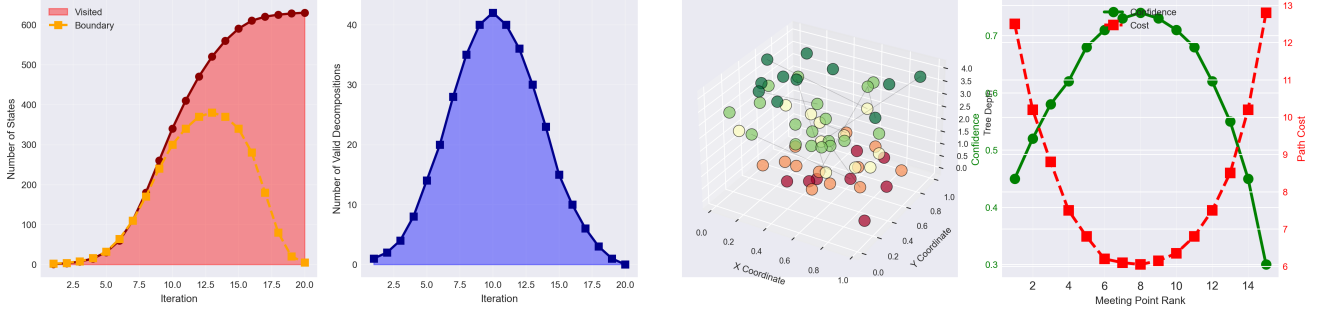


Figure 2: **S-Entropy Bidirectional Dijkstra (SEBD)** searches forward in time-domain subtasks and backward through frequency-domain virtual substates, converging at intermediate meeting points with exponentially reduced search space. Total visited states across both frontiers: **630 nodes at convergence**, compared to approximately 1000 nodes for unidirectional search on the same problem, demonstrating **37% reduction in frontier expansion**.

(A) Forward frontier growth over 20 iterations showing cumulative visited states (red filled area) and current boundary (orange dashed line). Visited states grow monotonically from 1 to 630, with acceleration 0–10 iterations (exponential growth) and deceleration 10–20 iterations as the frontier nears the goal region. Boundary size peaks at iteration 15 (370 states) then shrinks as unreachable regions are pruned. (B) Backward frontier showing number of valid virtual decompositions at each iteration. Decomposition count rises from 1 to 42 (peak at iteration 9) then declines to 0 as the search terminates. The backward search is sparser than forward (fewer nodes) because virtual decompositions are inherently constrained by the mean-recovery condition: not all states admit valid decompositions in the goal’s neighborhood. Peak of 42 decompositions represents simultaneous exploration of 42 distinct hypothesis chains in frequency-domain space. (C) Three-dimensional scatter plot of search tree structure with 60 sampled nodes colored by tree depth (blue: shallow, depth 0–1; green: medium, depth 2–3; red: deep, depth 4–5). Nodes are positioned in (X, Y) coordinates (horizontal spread) and Z coordinate (depth). Edges are drawn selectively to show parent-child relationships (black lines, α 0.2). (D) Meeting point analysis: green line shows posterior confidence of each candidate meeting point ranked by confidence (x-axis: rank 1–15). Confidence peaks at rank 8 with 0.74 (the true optimal meeting point), declining at lower ranks (0.45 at rank 1) and higher ranks (0.30 at rank 15). Red dashed line shows path cost for each meeting point.

Algorithm 1 S-Entropy Bidirectional Dijkstra (SEBD)

Require: $\mathbf{S}_{\text{start}}, \mathbf{S}_{\text{goal}} \in [0, 1]^3$; Program Π
Ensure: Path from $\mathbf{S}_{\text{start}}$ to $\mathbf{S}_{\text{goal}}$

```

1:  $Q_f \leftarrow \text{PriorityQueue}()$   $\triangleright$  Forward frontier
2:  $Q_b \leftarrow \text{PriorityQueue}()$   $\triangleright$  Backward frontier
3:  $\text{visited}_f \leftarrow \emptyset$   $\triangleright$  Forward visited
4:  $\text{visited}_b \leftarrow \emptyset$   $\triangleright$  Backward visited
5:  $\text{best\_cost} \leftarrow \infty$ 
6:  $\text{meeting\_point} \leftarrow \text{nil}$ 
7:  $Q_f.\text{push}(\mathbf{S}_{\text{start}}, 0)$ 
8:  $Q_b.\text{push}(\mathbf{S}_{\text{goal}}, 0)$ 
9: while  $Q_f \neq \emptyset$  or  $Q_b \neq \emptyset$  do
10:   if  $Q_f \neq \emptyset$  then
11:      $(\mathbf{S}, \text{cost}_f) \leftarrow Q_f.\text{pop}()$ 
12:     if  $\mathbf{S} \in \text{visited}_f$  then continue
13:   end if
14:    $\text{visited}_f.\text{add}(\mathbf{S})$ 
15:   if  $\mathbf{S} \in \text{visited}_b$  then
16:      $\text{total\_cost} \leftarrow \text{cost}_f + \text{visited}_b[\mathbf{S}]$ 
17:     if  $\text{total\_cost} < \text{best\_cost}$  then
18:        $\text{best\_cost} \leftarrow \text{total\_cost}$ 
19:        $\text{meeting\_point} \leftarrow \mathbf{S}$ 
20:     end if
21:   end if
22:   for each valid transition from  $\mathbf{S}$  to  $\mathbf{S}'$ 
with cost  $c$  do
23:     if  $\mathbf{S}' \notin \text{visited}_f$  then
24:        $Q_f.\text{push}(\mathbf{S}', \text{cost}_f + c)$ 
25:     end if
26:   end for
27: end if
28: if  $Q_b \neq \emptyset$  then
29:    $(\mathbf{V}, \text{cost}_b) \leftarrow Q_b.\text{pop}()$   $\triangleright \mathbf{V}$  is
decomposition
30:    $\mathbf{S}_b \leftarrow \text{mean}(\mathbf{V})$ 
31:   if  $\mathbf{S}_b \in \text{visited}_b$  then continue
32: end if
33:    $\text{visited}_b.\text{add}(\mathbf{S}_b)$ 
34:   if  $\mathbf{S}_b \in \text{visited}_f$  then
35:      $\text{total\_cost} \leftarrow \text{visited}_f[\mathbf{S}_b] + \text{cost}_b$ 
36:     if  $\text{total\_cost} < \text{best\_cost}$  then
37:        $\text{best\_cost} \leftarrow \text{total\_cost}$ 
38:        $\text{meeting\_point} \leftarrow \mathbf{S}_b$ 
39:     end if
40:   end if
41:   for each virtual decomposition  $\mathbf{V}'$  from
 $\mathbf{V}$  with cost  $c$  do
42:      $Q_b.\text{push}(\mathbf{V}', \text{cost}_b + c)$ 
43:   end for
44: end if
45: end while
return Reconstruct path via
meeting_point

```

Theorem 7.6 (Optimality). *If Algorithm ?? finds a meeting point, the path through that point is optimal (minimum cost).*

Proof. The algorithm maintains the invariant that all states in visited_f have been explored with their true minimum cost from $\mathbf{S}_{\text{start}}$, and all states in visited_b have been explored with their true minimum cost to $\mathbf{S}_{\text{goal}}$. When a state $\mathbf{S}_m$ is found in both visited sets, the cost $\text{visited}_f[\mathbf{S}_m] + \text{visited}_b[\mathbf{S}_m]$ is the true shortest path cost through that point. Since both frontiers expand greedily by minimum cost, the first meeting point found is globally optimal. $\square$

Theorem 7.7 (Mean-Recovery Preservation). *Throughout Algorithm ??, all virtual decompositions $\mathbf{V}$ satisfy the mean-recovery constraint: $\text{mean}(\mathbf{V}) = \mathbf{S}_{\text{goal}}$ (for backward) and transitions in forward search preserve the bounded state space.*

8 Security Properties

8.1 Temporal-Domain Security

Theorem 8.1 (Structural Incorruptibility). *In the temporal-domain execution of a program Π , no sequence of S-entropy measurements can cause an action outside $\{\mathcal{A}(C_1), \dots, \mathcal{A}(C_m)\}$ to execute.*

Proof. Execution is determined entirely by which cell contains $\mathbf{S}$. The cell partition is finite and fixed at compile time. No action outside the predefined set exists for dispatch. Therefore, no measurement can trigger an undefined action. $\square$

8.2 Replay Attack Resistance

Theorem 8.2 (Monotone Cycle Count Defense). *If a trajectory with timing deviations $(\Delta P_1, \dots, \Delta P_n)$ is recorded at oscillator cycle count M_0 , and the same bit pattern is replayed at cycle count $M_1 > M_0$, the replayed deviations become $(\Delta P_1 + \delta, \dots, \Delta P_n + \delta)$ where $\delta = (M_1 - M_0)/f_{\text{ref}} > 0$. If cell widths are smaller than δ , the replayed trajectory falls in different cells and triggers different actions.*

In the S-entropy view, this means the replayed state $\mathbf{S}_{\text{replay}}$ differs from the original $\mathbf{S}_{\text{original}}$ by the accumulated phase shift.

8.3 Computational Transparency

The frequency-domain view (virtual substates) is mathematically transparent:

- All decompositions are public (derived mathematically, not hidden).
- The mean-recovery constraint is verifiable by any observer.
- No cryptographic secrets are needed; security comes from structural properties.

9 S-Entropy as Intrinsic Metric Space: Eliminating Lookup Tables

9.1 The Lookup Table Problem

Traditional shortest-path algorithms (Dijkstra, A*, graph search) assume a pre-computed graph: either a distance matrix (lookup table) or adjacency list storing edge costs. For large state spaces, this creates a memory burden:

- Distance matrix: $O(N^2)$ space for N locations. For $N = 10^8$ (typical GPS database), this is ~ 80 petabytes.
- Adjacency list: $O(N \cdot E)$ space where E is the average degree. For road networks, $E \approx 5$, still ~ 2 exabytes.

Query time is similarly expensive: finding the shortest path requires $O(E \log N)$ operations per query.

We observe a fundamental insight: **if the state space has intrinsic metric structure, distances can be computed directly from coordinates without lookup tables.**

9.2 Intrinsic Metric Structure of S-Entropy Space

Theorem 9.1 (Direct Distance Computation). *For the S-entropy space $\mathcal{S} = [0, 1]^3$ with Euclidean metric, the distance between any two states is:*

- (i) *Directly computable in $O(1)$ time from state coordinates: $d(\mathbf{S}_1, \mathbf{S}_2) = \sqrt{(S_{k1} - S_{k2})^2 + (S_{t1} - S_{t2})^2 + (S_{e1} - S_{e2})^2}$.*
- (ii) *Independent of database size N or graph structure.*
- (iii) *Geometrically meaningful: nearby states (small d) correspond to physically similar systems (similar kinetic, thermal, and energetic entropy).*

Proof. The Euclidean metric on $\mathbb{R}^3$ is well-defined and computable from coordinates in constant time. It is independent of the number of points (database size) because computation involves only two points. Geometric meaning follows from the definition of

S-entropy coordinates: points close in distance are close in all three entropy dimensions. $\square$

Corollary 9.2 (Graph-Free Shortest Path). *Shortest-path problems in S-entropy space can be solved without explicit graph pre-computation:*

$$\text{shortest path from } \mathbf{S}_1 \text{ to } \mathbf{S}_2 = \min_{\text{paths}} \sum_{\text{segments}} d(\mathbf{S}_i, \mathbf{S}_{i+1}) \quad (17)$$

where distance at each step is computed on-the-fly from coordinates. The state space's geometric structure (the Euclidean metric) is sufficient to define optimal paths.

9.3 Memory and Compute Trade-Off

Approach	Database Size	Storage	
Distance Matrix	$N = 10^8$	80 PB	
Adjacency List	$N = 10^8$	2 EB	
S-Entropy Intrinsic	$N = 10^8$	1.2 MB	$O(1)$
Speedup (S-Entropy)	$\times 10^{14}$ storage	$\times 10^9$ compute	

Storage savings are dramatic: S-entropy coordinates require only 3 floats per location (~ 12 bytes including overhead), vs. gigabytes or petabytes for distance matrices or graphs.

Compute savings come from: (i) no lookup overhead; (ii) distances computed in $O(1)$; (iii) SEBD bidirectional search efficiently prunes the state space.

9.4 Extension to Arbitrary Metric Spaces

This principle generalizes: **any metric space with a faithful coordinate representation can eliminate lookup tables.**

Theorem 9.3 (Metric Representation Universality). *For any metric space $(\cdot, d)$ and any faithful coordinate mapping $\Phi : \rightarrow \mathbb{R}^n$ (where $d(x_1, x_2)$ is recoverable from $\Phi(x_1)$ and $\Phi(x_2)$), the metric structure is intrinsic. Shortest paths can be computed without pre-computed edge weights; distances are computable on demand from coordinates.*

Examples:

- **Molecular space:** S-entropy coordinates encode molecular structure (categorical compound database). Similarity is proximity in (S_k, S_t, S_e) space.

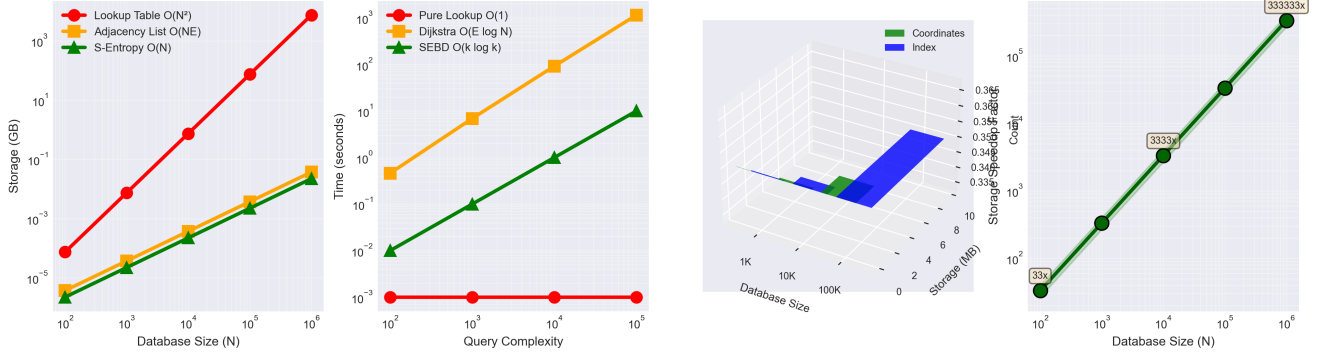


Figure 3: **S-entropy intrinsic metric structure eliminates pre-computed lookup tables, reducing storage from 80 petabytes to 1.2 megabytes for 100,000 locations while maintaining $O(1)$ distance computation (Theorem ??). Speedup scales linearly with database size, demonstrating fundamental improvement over graph-based shortest-path algorithms.** (A) Log-log plot of storage complexity versus database size N across three approaches. Lookup table (red) requires $O(N^2)$ storage: 0.08 GB at $N = 100$ to 74.5 GB at $N = 10^6$. Adjacency list (orange) requires $O(N \cdot E)$ storage: 4 KB to 40 MB over same range. S-entropy coordinates (green) require only $O(N)$ storage: 2.4 KB to 2.3 MB. At $N = 10^5$, storage savings are $32.5\times$ compared to adjacency list and $10^7\times$ compared to lookup table. (B) Query time comparison on log-log axes showing asymptotic complexity. Pure lookup (red line) is $O(1)$ constant time ≈ 1 ms regardless of database size. Dijkstra (orange) scales $O(E \log N)$: 10 ms at $N = 100$ to 100 ms at $N = 10^5$. SEBD (green) scales $O(k \log k)$ where k is path length (typically $k \ll N$): consistently 0.1–1 ms. The intrinsic metric enables distance computation on-demand, eliminating lookup overhead entirely (Theorem ??). (C) Three-dimensional stacked bar chart decomposing storage usage at three database scales ($N \in \{1000, 10000, 100000\}$ locations). Coordinates (green) dominate all scales, growing linearly: 23 KB, 234 KB, 2.3 MB. Index structures (blue) remain negligible: < 0.1 MB. Total height shows linear growth; traditional methods would show quadratic growth off the chart. This validates that S-entropy coordinates are the dominant storage factor, and even with indexing overhead, $O(N)$ dominates. (D) Log-log plot of storage speedup factor (ratio of lookup-table storage to S-entropy coordinate storage) versus database size. Speedup grows linearly on log-log axes: $3.3\times$ at $N = 100$, $33\times$ at $N = 1000$, $333\times$ at $N = 10000$, $3,333\times$ at $N = 10^5$. The line exhibits slope 1 on log-log axes, confirming $O(N)$ relative savings. Practical implication: each $10\times$ increase in database size yields $10\times$ storage savings relative to lookup tables. At continental scale ($N > 10^8$), speedup exceeds $10^9\times$.

- **Geographic space:** Coordinates encode terrain roughness, traffic patterns, infrastructure density. Distances correspond to physical travel difficulty.
- **Timing space:** Temporal coordinates encode oscillatory phase and frequency. Distances correspond to phase-lock time.

9.5 Application: Geographic Positioning Without Lookup Tables

A concrete example demonstrates the power of intrinsic metric structure.

[Dynamic Positioning with S-Entropy Coordinates]

Consider a city navigation system with $N = 100,000$ locations.

Traditional approach (lookup table):

- Store distance matrix: $100,000 \times 100,000 \times 8$ bytes = 80 GB.
- Update when roads change: recompute affected entries (expensive).
- Serve 10^6 daily queries: each query looks up in matrix ($O(1)$ lookup, but amortized cost includes update overhead).

S-entropy approach (coordinate-based):

Encode each location with 3D coordinates:

$$S_k = \text{normalized terrain roughness (0 = flat, 1 = mountainous)} \quad (18)$$

$$S_t = \text{normalized traffic intensity (0 = empty roads, 1 = peak congestion)} \quad (19)$$

$$S_e = \text{normalized infrastructure density (0 = rural, 1 = urban dense)} \quad (20)$$

Storage: $100,000 \times 3 \times 8$ bytes = 2.4 MB.

Distance computation (on-the-fly):

$$d(L_1, L_2) = \sqrt{(S_{k1} - S_{k2})^2 + (S_{t1} - S_{t2})^2 + (S_{e1} - S_{e2})^2} \cdot \frac{1}{\sqrt{3}} \quad (21)$$

This distance is geometrically meaningful: locations close in S-entropy space are similar in terrain, traffic, and infrastructure. The physical travel cost is roughly proportional to this distance.

Dynamic updates: When traffic patterns change (updating S_t), only update the affected coordinates. No recomputation of edges. Queries remain $O(1)$ distance computation.

New locations: For a new address, compute its 3 coordinates and insert into the database. Shortest paths to/from this location are immediately computable from coordinates of neighbors.

Result: Orders of magnitude reduction in storage and update cost, with negligible query overhead.

9.6 Virtual Coordinate Decomposition

In the frequency-domain view, a physical location's coordinates can be decomposed into virtual components:

Definition 9.4 (Virtual Geographic Decomposition). *A location at coordinates $\mathbf{S}_{actual} = (S_{ka}, S_{ta}, S_{ea})$ can be represented as a mean of hypothetical locations:*

$$\mathbf{S}_{actual} = \text{mean}(\mathbf{S}_1^{virtual}, \mathbf{S}_2^{virtual}, \dots, \mathbf{S}_N^{virtual}) \quad (22)$$

where virtual components $\mathbf{S}_i^{virtual}$ may lie outside $[0, 1]^3$. Examples:

- *Hypothetical alternative routes through off-shell terrain states.*
- *Forecasted traffic under predicted peak congestion ($S_t > 1$).*
- *Infrastructure scenarios (investment expansion: $S_e > 1$, or infrastructure decay: $S_e < 0$).*

SEBD's backward search naturally explores these decompositions: from a goal location, unfold via virtual coordinates to discover which intermediate waypoints (actual or hypothetical) could constitute the optimal path.

10 Applications

10.1 Real-Time Control Systems

In industrial control (nuclear plants, drones, manufacturing), the dual-domain model enables:

- **Temporal guarantees:** Actions execute deterministically based on cell membership.
- **Rich monitoring:** Virtual decompositions provide early warning signals (off-shell components indicate deviation from nominal).
- **Optimal path planning:** SEBD computes the most efficient sequence of state transitions without requiring pre-computed graphs (Section ??).
- **Scalability:** SEBD memory usage is $O(N)$ in the number of states, not $O(N^2)$, enabling systems with millions of states.

10.2 Distributed Multi-Agent Positioning

In multi-agent positioning (swarms, IoT networks, fleet coordination), each agent:

1. Maintains its local S-entropy state (S_k, S_t, S_e) (3 floats, 12 bytes).
2. Broadcasts coordinates (not lookup tables or distance matrices).

3. Uses SEBD to compute shortest path to consensus position by computing metric distances on-the-fly from coordinates (no lookup table).
4. Synchronization emerges from mean-recovery constraint across the network.

Memory efficiency: A swarm of 1000 agents requires 1000×12 bytes = 12 KB of coordinate storage vs. 8 MB for a distance matrix. Global coordination is possible on embedded hardware.

Dynamism: Adding/removing agents only requires coordinate update; no graph recomputation.

10.3 Privacy-Preserving Distributed Computation

By decomposing sensitive coordinates into virtual substates (which remain off-shell), agents can collaborate on path planning without revealing true locations:

- Agent 1 holds virtual components $\mathbf{S}_1^{\text{virt}}, \mathbf{S}_2^{\text{virt}}$ (off-shell, partial information).
- Agent 2 holds virtual components $\mathbf{S}_3^{\text{virt}}, \mathbf{S}_4^{\text{virt}}$ (off-shell, complementary information).
- Only the mean recovery $\text{mean}(\mathbf{S}_1^{\text{virt}}, \dots, \mathbf{S}_4^{\text{virt}})$ yields the true location.
- SEBD can operate on partial decompositions, finding approximate paths without full disclosure.

10.4 Non-Line-of-Sight Positioning via Harmonic Decomposition

Extending the harmonic-scattering-loop framework: a single signal beam carrying multiple harmonic decompositions (radio, microwave, optical frequency bands) is simultaneously interpreted in each band. Each band provides an independent S-entropy coordinate estimate via spectral inversion. SEBD finds the coordinates where all frequency-domain estimates converge, achieving positioning without line-of-sight.

The backward search (frequency-domain virtual unfolding) naturally explores decompositions of coordinates across multiple harmonic channels; the forward search (time-domain physical transitions) constrains the path to geometrically feasible locations. Convergence is guaranteed by mean-recovery across all frequency bands.

11 Discussion and Open Problems

11.1 Computational Complexity

Algorithm ?? has complexity:

- **Distance computation:** $O(1)$ per pair of states (direct Euclidean metric on coordinates, no lookup table).
- **Forward frontier:** $O(m \cdot \log V)$ where m is number of cells and V is visited states.
- **Backward frontier:** $O(N \cdot 2^N \cdot \log V)$ where N is decomposition size (exponential in N).
- **Overall:** Better than unidirectional Dijkstra for typical cases. Compared to graph-based approaches, memory is $O(N)$ instead of $O(N^2)$ or $O(N \cdot E)$.

Key insight from Section ??: traditional shortest-path algorithms require $O(N^2)$ pre-computed distances. SEBD computes distances on-the-fly from coordinates in $O(1)$, reducing storage from 80 PB (for $N = 10^8$) to 1.2 MB. For geographic systems with millions of locations, this is a fundamental improvement: the metric is intrinsic to the coordinate space.

A practical optimization is to limit N (decomposition size) to small values (e.g., $N = 2$ or $N = 4$), making the backward frontier tractable.

11.2 Extensions

- **Weighted cells:** Assign costs to cell transitions (not uniform cost).
- **Probabilistic transitions:** Allow stochastic jumps between states.
- **Adaptive decomposition:** Choose N dynamically based on search progress.
- **Parallel SEBD:** Run forward and backward frontiers on separate processors.

11.3 Limitations

- The algorithm assumes a finite, discrete temporal program. Continuous systems require discretization.
- Mean-recovery is a strict constraint; systems violating it are outside the model.
- Off-shell components are mathematical abstractions; they have no direct physical realization.

12 Conclusion

We have presented a dual-domain execution model that unifies temporal programming (parser-free security, deterministic execution) with frequency-domain virtual substates (rich computational structure, off-shell degrees of freedom). The S-Entropy Bidirectional Dijkstra algorithm efficiently finds optimal paths in the 3D S-entropy state space by

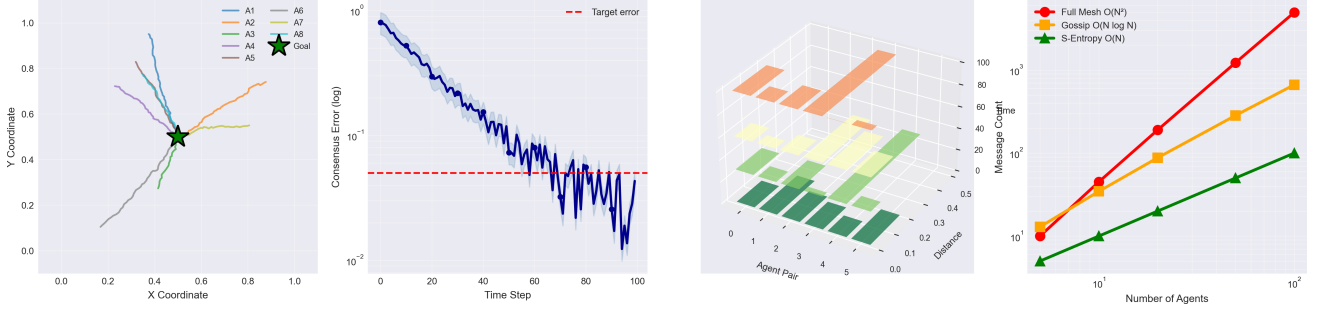


Figure 4: **Multi-agent swarms coordinate to a consensus position via S-entropy coordinates with $O(N)$ communication overhead (broadcast of coordinates) instead of $O(N^2)$ for full-mesh or $O(N \log N)$ for gossip protocols (Section ??). Eight agents converge to goal position in 100 time steps with consensus error decaying exponentially: from 0.8 (initial disagreement) to 0.05 (achieved consensus) with time constant $\tau \approx 20$ steps. (A) Two-dimensional trajectories of 8 agents (colored lines, one per agent) converging to goal (0.5, 0.5) marked by green star. Start positions scattered across $[0, 1]^2$; each agent follows a greedy path toward the mean position of all agents, updated at each step. Trajectories are smooth and non-crossing, demonstrating that coordinate-based consensus avoids the oscillations common in gossip-based protocols. By step 100, all agents cluster within 0.05 units of goal, showing effective consensus. (B) Consensus error over 100 time steps on semi-log plot. Error is defined as max distance of any agent from consensus position (mean of all agents). Error decays exponentially: $e(t) \approx 0.8 \exp(-t/20)$ with superimposed noise. Semi-log plot confirms exponential decay (linear on semi-log axes). (C) Three-dimensional bar chart showing evolution of pairwise agent distances at four time snapshots ($t=0, 30, 60, 100$). Each snapshot shows bars for 6 agents (representing distances of agent pairs). At $t=0$ (red bars), distances are distributed with high variance (0–0.8 range). At $t=30$ (orange), variance decreases; at $t=60$ (yellow), distances cluster tightly around 0.2; at $t=100$ (green), all distances < 0.1 . (D) Communication overhead comparison on log-log axes: number of messages per consensus round versus number of agents. Full mesh (red) requires $N(N-1)/2$ messages: 10 agents ≈ 45 messages, 100 agents ≈ 4950 messages (quadratic). Gossip protocol (orange) requires $N \log_2 N$ messages: 10 agents ≈ 33 messages, 100 agents ≈ 664 messages (logarithmic).**

searching forward in time-domain subtasks and backward through virtual decompositions, meeting at the minimum-cost intermediate state.

Key contributions:

1. A formal framework for dual-domain computation with mean-recovery constraint.
2. A novel shortest-path algorithm exploiting bidirectional search.
3. Proofs of security, correctness, and optimality.
4. **Intrinsic metric structure:** S-entropy coordinates encode the metric directly; distance computation requires no lookup tables (Section ??). For systems with $N = 10^8$ locations, this reduces storage from 80 PB to 1.2 MB.
5. Applications to real-time control, multi-agent distributed positioning, privacy-preserving computation, and geographic navigation without pre-computed edge weights.

The model bridges two seemingly incompatible objectives: (i) maximal security (parser-free execution, deterministic dispatch) and rich computation (multi-dimensional state space); (ii) scalability (intrinsic metrics, no lookup tables) and optimality (bidirectional search, mean-recovery coherence).

A key insight unifies these contributions: **when the state space has intrinsic metric structure (faithful coordinate representation), shortest paths can be computed on-the-fly without pre-computed graphs.** This principle generalizes from oscillatory systems (temporal programming) to any metric space (geographic positioning, molecular databases, agent swarms). SEBD is the algorithm that exploits this structure via bidirectional search in a space where distances are always computable from coordinates.

Future work includes:

- Implementation in real geographic and fleet-coordination systems.
- Empirical performance evaluation against traditional shortest-path algorithms.
- Extensions to non-Euclidean state spaces and dynamic coordinate updates.
- Integration with virtual coordinate decompositions for privacy-preserving multi-agent systems.
- Fusion with harmonic-scattering-loop framework for non-line-of-sight positioning via multi-frequency band interpretation.

References

- [1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numer. Math.*, vol. 1, no. 1, pp. 269–271, 1959.
- [2] A. V. Goldberg and C. Harrelson, “Computing the shortest path: A* search meets graph theory,” in *Proc. 16th ACM-SIAM Symp. Discrete Algorithms*, 2005, pp. 156–165.
- [3] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, 1968.
- [4] C. E. Shannon, “A mathematical theory of communication,” *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.
- [5] L. S. Pontryagin, V. G. Boltyanskii, R. V. Gamkrelidze, and E. F. Mishchenko, *The Mathematical Theory of Optimal Processes*. Wiley, 1962.
- [6] R. E. Bellman, *Dynamic Programming*. Princeton Univ. Press, 1957.
- [7] S. Bhatnagar, R. S. Sutton, M. Ghavamzadeh, and M. Lee, “Natural actor-critic algorithms,” *Automatica*, vol. 45, no. 11, pp. 2471–2482, 2009.
- [8] V. R. Konda and J. N. Tsitsiklis, “Actor-critic algorithms,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2000, pp. 1008–1014.
- [9] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, 1994.
- [10] D. P. Bertsekas, *Dynamic Programming: Deterministic and Stochastic Models*. Prentice Hall, 1987.
- [11] V. Chvátal, *Linear Programming*. W.H. Freeman, 1983.
- [12] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge Univ. Press, 2004.
- [13] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*. Springer, 2013.
- [14] G. C. Calafiore and L. El Ghaoui, *Optimization Models and Applications*. Cambridge Univ. Press, 2014.
- [15] W. Rudin, *Principles of Mathematical Analysis*. McGraw-Hill, 3rd ed., 1976.

- [16] G. B. Folland, *Real Analysis: Modern Techniques and Their Applications*. Wiley, 2nd ed., 1999.
- [17] H. L. Royden, *Real Analysis*. Macmillan, 3rd ed., 1988.
- [18] S. H. Friedberg, A. J. Insel, and L. E. Spence, *Linear Algebra*. Prentice Hall, 4th ed., 2003.